Recursive Co-Occurrence Blocking with Pair-Adaptive Similarity for Unsupervised Entity Resolution

Lou Angela Foua, Muzakkiruddin Ahmed Mohammed and Dr. John R. Talburt
Center for Advanced Research in Entity Resolution and Information Quality
University of Arkansas at Little Rock
Little Rock, Arkansas, USA
{ldfoua1, mmohammed6, jrtalburt}@ualr.edu

***Abstract*—Entity resolution identifies records that refer to the same real-world entity, but exhaustive comparison requires quadratic work. Blocking reduces this cost, although fixed blocking keys often trade true-match coverage for candidate precision. This paper presents a symbolic, unsupervised framework that combines recursive co-occurrence blocking with pair-adaptive meta-blocking. Starting from token-based blocks, the method extends a block key with locally frequent co-occurring tokens under a depth-dependent frequency floor. The resulting blocks are optionally reconciled, filtered by size or refinement-aware criteria, and converted into a weighted candidate graph. Each edge combines block-level evidence with the Jaccard similarity of the corresponding record pair, allowing two pairs induced by the same block to receive different weights. Existing experimental logs cover 185 evaluated settings on each of eight person-record datasets, for 1,480 recorded evaluations. The observed best setting varies by dataset. On the poor-data-quality S8P dataset, increasing the pair-adaptation weight from 0 to 0.75 raises F1 from 0.0662 to 0.4084, while a lower graph threshold yields the best observed F1 of 0.5281. Recursive depth mainly changes precision and runtime, with limited F1 gains beyond shallow refinement. Comparison with the Data Washing Machine (DWM), a prior unsupervised system that adds explicit all-pairs pairwise scoring on the same datasets, shows that the present blocking-only pipeline closes most of the quality gap on good-quality data (mean F1 difference of -0.025 across six datasets) while falling substantially behind on poor-quality data where DWM's edit-distance scoring is critical. These results support recursive block construction and pair-specific edge weighting as interpretable components for label-free entity-resolution pipelines, and identify pairwise signal integration as the primary direction for improvement on noisy data.**

***Index Terms*—entity resolution, record linkage, blocking, meta-blocking, adaptive similarity**

I. Introduction

Entity resolution (ER) determines whether records refer to the same person, organization, product, or other real-world entity. The problem originated in statistical record linkage and remains central to data integration because names, addresses, identifiers, and other attributes are frequently incomplete or inconsistent [1]–[3]. For N records, exhaustive comparison requires $\binom{N}{2}$ pair evaluations, which quickly becomes impractical. Blocking therefore precedes most ER systems: only record pairs sharing a candidate region are scored downstream. A related predecessor system, the Data Washing Machine (DWM), applies frequency-based blocking followed by exhaustive pairwise scoring within each block using a multi-token scoring matrix [4]. DWM is evaluated on several of the same person-record datasets used here, providing a direct empirical reference point for the present blocking-only approach.

Blocking introduces a precision–recall dilemma. A restrictive key may remove true matches before scoring, whereas permissive blocking may generate so many noisy pairs that it approaches all-pairs comparison. Classical sorted-neighborhood and token-blocking methods use fixed keys or fixed neighborhoods [5]–[7]. Redundancy-positive blocking and meta-blocking retain overlapping blocks and then prune the induced comparison graph [8], [9]. These approaches are effective, but two limitations motivate this work. First, an initially broad token block may contain locally informative token combinations that are not represented by the original key. Second, block-level weighting gives the same contribution to every pair in a block even when the records themselves have very different token overlap.

We address these limitations with recursive co-occurrence blocking and pair-adaptive edge weighting. Within each current block, locally frequent tokens extend the blocking key, creating progressively more specific candidate regions. After refinement and filtering, an ARCS-style graph is constructed. Its block contribution is modulated for each record pair by Jaccard similarity, so a generic shared block contributes less when the pair has little direct overlap. A thresholded graph is clustered by connected components, with an optional density-based splitting step.

The paper makes three contributions. First, it formulates symbolic blocking as a depth-limited local co-occurrence refinement process. Second, it introduces a pair-adaptive weighting term that interpolates between block-only evidence and fully pair-modulated evidence. Third, it analyzes the existing 1,480-run parameter study across eight person-record datasets, showing strong sensitivity to the pair-adaptation weight, graph threshold, and data-quality regime. The algorithm itself consumes no labeled match pairs. Ground truth is used only for evaluation; per-dataset best settings are reported as observed oracle upper bounds rather than as a label-free deployment rule.

II. Related Work

Early record-linkage research formalized matching as evidence aggregation under uncertainty [1], [2]. Blocking later became a standard efficiency stage, with the merge/purge and sorted-neighborhood paradigms limiting comparisons to nearby or key-sharing records [5]. Adaptive sorted neighborhoods vary the local window rather than applying one global size [6], [7], while learned blocking induces predicates or representations from training evidence [10].

Meta-blocking is the closest foundation for this work. It transforms redundant blocks into a weighted record graph and prunes weak comparisons [8]. Subsequent surveys organize blocking techniques around schema awareness, redundancy, filtering, and graph pruning [9]. The proposed method preserves the interpretable graph abstraction but changes both the block set and the edge evidence: block keys grow from local co-occurrence, and pairs within the same block are differentiated by their own token overlap.

Modern methods learn representations for blocking and entity matching, including schema-agnostic web-entity resolution, AutoBlock, DeepBlocker, and self-supervised data-integration models [11]–[14]. These methods can provide strong retrieval quality but may require training, learned embeddings, or additional infrastructure. The present approach instead targets a transparent symbolic regime in which every candidate edge can be traced to retained block keys and direct token overlap.

A directly relevant unsupervised predecessor is the Data Washing Machine (DWM) [4]. DWM uses frequency-based token blocking to group candidates, then scores every pair within a block using a Monge-Elkan-style multi-token scoring matrix with normalized Damerau-Levenshtein similarity; accepted clusters are kept while rejected ones are reprocessed at a higher match threshold, regulated by a Shannon entropy quality criterion. This pipeline involves $O(\sum_B |B|^2/2)$ pairwise scoring operations over the blocks. The present method removes the explicit pairwise scoring step: pair-adaptive weighting in (6) modulates block contributions by Jaccard token overlap computed during graph construction, avoiding a separate all-pairs matching pass. Comparing the two systems on shared datasets shows how much quality can be recovered from blocking evidence alone versus a full pairwise scoring pipeline.

III. Proposed Method

A. End-to-End Pipeline

The framework is a deterministic sequence of record, block, and graph transformations. Fig. 1 summarizes the data flow. Token normalization and document-frequency filtering reduce generic evidence. Initial blocks provide broad candidate coverage; recursive refinement then creates local token conjunctions. Reconciliation and top-k filtering control the resulting block collection. Finally, the retained blocks induce an adaptively weighted graph whose thresholded components define entities.

The design separates candidate generation from pair discrimination. Recursive blocking determines which pairs can be considered, whereas (6) determines how strongly each retained block supports a particular pair. This separation is useful for interpretation: a predicted link can be traced to its block keys, direct token overlap, threshold, and graph component.

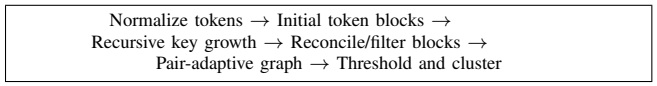


Fig. 1. Overview of the recursive blocking and adaptive weighting pipeline.

B. Representation and Initial Blocking

Let $R = \{r_1, \ldots, r_N\}$ be a record collection and let T_i be the normalized token set of record r_i. Text is uppercased, split into tokens, and stripped of non-word characters. Very high-document-frequency tokens are removed before blocking. A block is

$$B_K = \{r_i : K \subseteq T_i\}, \tag{1}$$

where K is a token-key tuple. Default blocking retains tokens satisfying document-frequency, minimum-length, and optional numeric constraints. A full redundancy-positive mode instead creates a block for every token occurring in at least two records. The implementation also contains a hierarchical mode, but the main method uses token blocks as the starting point for recursive refinement.

C. Recursive Co-Occurrence Refinement

For each block B_K, the method counts tokens not already in K. At recursion depth d, token t produces a child block when its local count meets the depth-coupled floor $\mu_d = \max(d, \mu_{\min})$:

$$B_{K\cup\{t\}} = \{r_i \in B_K : t \in T_i\}, \quad c_{B_K}(t) \geq \mu_d. \tag{2}$$

Every child satisfies $K \subset K \cup \{t\}$ and $B_{K\cup\{t\}} \subseteq B_K$. Thus, recursion grows the key while narrowing the record set. The increasing floor limits accidental combinations at deeper levels, and recursion stops at a maximum depth, an empty refinement, or a fixed point.

For example, a broad *SMITH* block can be refined by a locally frequent *OAK* token, isolating the records that share both terms. The refined child supplies selective block evidence, while pair adaptation can still down-weight a child-block pair with weak overall token overlap.

Recursive paths can create duplicate or nested record sets. The implementation optionally merges blocks with identical memberships and optionally applies a record-set containment purge. The merge operator indexes blocks by their record-ID sets and combines keys when two recursive paths select the same records. This preserves the membership set while reducing duplicate block objects, although combining keys can alter any later feature that depends on key structure. The purge operator removes a block when its membership is a strict subset of another retained membership. Such a purge preserves unweighted candidate-pair coverage only when the superset remains, but it can remove refined evidence used by key-aware filtering or graph weighting. For that reason, merge and purge are exposed independently and are treated as explicit ablation factors rather than described as universally lossless.

D. Refinement-Aware Block Filtering

After refinement, each record is retained only in its top-k blocks. The classical size mode prefers smaller blocks. Two refinement-aware alternatives exploit the key structure:

$$q_{\text{spec}}(B_K) = \frac{|B_K|}{|K|}, \tag{3}$$

$$q_{\text{key}}(B_K) = (-|K|, |B_K|). \tag{4}$$

The first is a specificity cost, so smaller values are preferred; the second prioritizes longer keys and breaks ties by block size. A composite mode additionally combines normalized block size, key length, and mean token length. Blocks whose retained membership falls below the minimum block size are removed.

E. Pair-Adaptive Meta-Blocking

The retained blocks induce a weighted graph $G = (V, E, w)$ over records. The implemented block contribution is

$$a(B) = \begin{cases} 1/|B|, & \text{uniform,} \\ \log(N/|B|)/|B|, & \text{IDF-style.} \end{cases} \tag{5}$$

Optional key-type, token-length, and intra-block-density factors can multiply $a(B)$. The central pair-adaptive term is

$$w_{ij} = \sum_{B \ni i,j} a(B) \left[(1-\lambda) + \lambda J(T_i, T_j)\right], \tag{6}$$

where $J(T_i, T_j) = |T_i \cap T_j|/|T_i \cup T_j|$ and $\lambda \in [0, 1]$. At $\lambda = 0$, pairs receive block-only evidence. At $\lambda = 1$, every block contribution is fully modulated by direct token overlap. Intermediate values preserve part of the broad-block evidence while reducing weights for weak pairs.

Edges with $w_{ij} \geq \tau$ are retained and clustered using Union-Find connected components. An optional density floor examines components of at least a configured size; low-density components are split by removing weak internal edges until disconnected subcomponents emerge. This is a greedy refinement heuristic intended to reduce bridge-induced chain collapse.

Let $M = \sum_i |T_i|$ denote total token occurrences. Normalization and document-frequency counting are $O(M)$. One refinement level costs $O(\sum_B \sum_{r_i \in B} |T_i|)$ before duplicate-key aggregation. The dominant graph cost is $O(\sum_B \binom{|B|}{2})$ over retained blocks, although duplicate pairs can be accumulated across blocks. Union-Find clustering is near-linear in the retained edge count. A pair-cost guardrail can drop oversized blocks before graph expansion. The implementation logs block counts, candidate-related counters, recursive time, filtering time, clustering time, and the exact invocation, allowing cost changes to be associated with individual stages rather than only with end-to-end runtime.

IV. Experimental Setup

A. Datasets and Evaluation Protocol

The existing evaluation uses eight tokenized person-record datasets containing names, addresses, cities, states, ZIP codes, and identifier fields. Table I summarizes the supplied corpus statistics. The G and P markers denote good- and poor-data-quality truth regimes, respectively; X marks additional schema noise.

TABLE I
Evaluated Person-Record Datasets

Data	Records	Clusters	Mean size	DQ
S1G	50	30	1.67	Good
S2G	100	62	1.61	Good
S4G	1,912	1,188	1.61	Good
S5G	3,004	1,877	1.60	Good
S7GX	2,912	1,827	1.59	Good/X
S8P	1,000	195	5.13	Poor
S12PX	6,000	693	8.66	Poor/X
S14GX	5,000	2,183	2.29	Good/X

TABLE II
Representative Parameter-Sensitivity Axes

Axis	Evaluated values
Blocking mode	Default, full, hierarchical
Filter mode	Size, specificity, keyLen, composite
Recursion depth	1, 2, 3, 5, 7, 10
Pair weight λ	0, .25, .50, .75, 1
Edge threshold τ	.05, .10, .20, .30, .40, .50
Top-k	1, 2, 3, 5, 10
Density floor δ	0, .30, .50, .70
Minimum block size	2, 3, 4, 5

An automated harness records 185 evaluated parameter/ablation cells per dataset, totaling 1,480 logged evaluations. The sweeps cover merge/purge behavior, filter mode, composite-filter weights, edge threshold τ, top-k, uniform versus IDF-style block weighting, blocking mode, recursion depth, density floor, pair-adaptation weight λ, and minimum block size. Representative axes are listed in Table II. Each run stores the predicted clustering, pairwise metrics, command line, timing, and stage diagnostics. No additional experiments are introduced in this paper; all reported numbers come from those existing logs.

The analysis addresses three questions: (RQ1) how pair adaptation and the edge threshold change the precision–recall operating point; (RQ2) whether recursion depth and refinement-aware filtering improve final F1; and (RQ3) whether one observed setting is competitive across different data-quality regimes. Pairwise precision, recall, and F1 are computed from predicted and ground-truth clusters. The displayed per-dataset maximum is selected with ground-truth F1 and is therefore an oracle description of the observed parameter surface, not an unsupervised tuning procedure. Parameter sweeps are interpreted as sensitivity analyses under fixed companion settings and merge/purge cells.

For label-free comparison of ER outputs, the Case Count Metric System classifies cluster changes as unchanged, merged, partitioned, or overlapping [15].

The harness stores cluster outputs, structured metrics, invocation manifests, logs, and wall-clock measurements, which aggregation scripts convert into per-dataset reports. The pipeline has no stochastic training stage; automation is treated as a reproducibility aid rather than an algorithmic contribution.

TABLE III
BEST OBSERVED RESULT PER DATASET (ORACLE SELECTION)

Data	Prec.	Rec.	F1	Best logged setting
S1G	.964	1.000	.9818	$\tau = .30$, no-merge
S2G	.917	.917	.9167	Full, no-merge
S4G	.941	.831	.8826	$\delta = .70$, merge-only
S5G	.873	.900	.8861	Full, merge-only
S7GX	.871	.903	.8863	Full, merge-only
S8P	.553	.506	.5281	$\tau = .05$, merge-only
S12PX	.664	.306	.4185	$\tau = .05$, merge-only
S14GX	.923	.839	.8788	Full, merge-only

B. Code and Data Availability

The implementation, experiment runners, aggregate result workbooks and reports, retained S8P run artifacts, synthetic example, manuscript sources, and reproducibility documentation are publicly available in the Recursive ER Research Artifact repository.[1] The raw person-record datasets used for the reported evaluation are not redistributed because their provenance, privacy status, and redistribution rights remain under review; the repository provides the expected input schemas and instructions for running the pipeline with authorized local copies.

V. RESULTS AND DISCUSSION

A. Observed Best Settings

Table III reports the highest logged F1 for each dataset. No single setting dominates the eight datasets. The small S1G corpus is nearly solved by adjusting τ, whereas S2G, S5G, S7GX, and S14GX favor full blocking. The two poor-quality datasets require $\tau = 0.05$, consistent with their larger truth clusters and need for weaker connecting edges. S4G is the only dataset whose best logged setting is determined by the density-floor sweep.

The table also shows that dataset difficulty varies substantially. All six good-quality datasets exceed .87 best observed F1; by contrast, S8P and S12PX reach .5281 and .4185. Six of the eight oracle-best rows use the merge-only cell, but this is descriptive rather than a controlled merge ablation because the winning rows arise from different parameter groups. These results should not be interpreted as evidence that one globally deployable configuration has been found. Rather, they show that blocking permissiveness and graph pruning interact with data quality and truth-cluster structure.

B. Pair Adaptation and Edge Threshold

Table IV isolates the pair-adaptation sweep on S8P for the merge-only cell. Block-only weighting ($\lambda = 0$) yields high recall but extremely low precision. Moving to $\lambda = .50$ raises F1 from .0662 to .3940, and $\lambda = .75$ produces the best value in this sweep, .4084. Full modulation ($\lambda = 1$) raises precision further but loses too much recall. Thus, the pair term in (6) is not simply a monotonic quality control; it moves the operating point from permissive block evidence toward stricter direct similarity.

[1] https://github.com/MohdMuzakkiruddinAhmed/Recursive-ER-Research-Artifact

TABLE IV
PAIR-ADAPTATION SWEEP ON S8P (MERGE-ONLY)

λ	Precision	Recall	F1
0.00	.0349	.6450	.0662
0.25	.0636	.6069	.1151
0.50	.3134	.5304	.3940
0.75	.7416	.2818	**.4084**
1.00	.8379	.1526	.2582

TABLE V
EDGE-THRESHOLD SWEEP ON S8P (MERGE-ONLY)

τ	Precision	Recall	F1
.05	.553	.506	**.528**
.10	.696	.284	.403
.20	.823	.153	.259
.30	.930	.094	.171
.50	.988	.029	.056

The threshold sweep on the same dataset shows a complementary trade-off (Table V). The low threshold is beneficial because poor-quality truth clusters have mean sizes above five, so transitive recovery depends on retaining medium-weight edges. Increasing τ steadily raises precision but removes the edges required to recover larger components. On S4G, however, a density floor of .70 produces the highest observed F1 (.8826), indicating that aggressive splitting can help when clusters are smaller and bridge errors are more damaging.

C. Refinement Depth, Filtering, and Blocking Mode

On S8P, the depth sweep changes recursive runtime from .086 s at depth 1 to 2.187 s at depth 10, while F1 remains between .214 and .223 (Table VI). Precision generally rises from .861 to .889 but is not strictly monotonic. The result supports shallow refinement as a practical default: deeper recursion changes block specificity and cost more than final F1 under this setting.

The matched S8P filter-mode sweep produces F1 values of .257 for size, .263 for specificity cost, .265 for key length, and .259 for the composite rule. The improvement is modest, but it supports the premise that key depth can carry information not represented by block cardinality alone. Full blocking is the best logged blocking mode on four datasets, while hierarchical blocking is not best on any dataset and yields .092 F1 on S8P. The full mode improves candidate coverage but increases cost; on S12PX, the supplied logs report 17.5 s versus 1.0 s for the cited default depth-1 comparison.

These observations also qualify the role of recursion. Refinement is not a uniformly improving matching operator; it changes the candidate evidence available to filtering and graph weighting. In the displayed S8P depth sweep, later stages largely absorb those changes, leaving final F1 nearly flat. Its strongest empirical role in the present logs is therefore interpretability and precision control rather than a large standalone F1 increase.

TABLE VI
RECURSIVE-DEPTH SWEEP ON S8P (NO-MERGE)

Depth	Precision	Recall	F1	Time (s)
1	.861	.128	.223	.086
2	.858	.127	.221	.442
3	.875	.122	.214	.826
5	.883	.124	.217	1.375
7	.874	.128	.223	1.881
10	.889	.123	.216	2.187

TABLE VII
FILTER AND DENSITY SWEEPS ON S8P (MERGE-ONLY)

Setting	Precision	Recall	F1
Size filter	.819	.153	.257
Specificity filter	.811	.157	.263
Key-length filter	.803	.158	**.265**
Composite filter	.824	.153	.259
$\delta = .00$	.828	.153	.258
$\delta = .30$	.823	.153	**.259**
$\delta = .50$	.840	.151	.256
$\delta = .70$	.912	.104	.186

D. Filter and Density Sensitivity

Table VII summarizes two additional S8P sweeps under the reported merge-only cell. Refinement-aware filters produce small F1 gains over size-only ranking, whereas the density floor mainly trades recall for precision. At $\delta = .70$, precision increases to .912 but recall falls to .104. This behavior is consistent with the purpose of density splitting: it removes weak graph bridges and therefore cannot recover links that are already absent. Its value is dataset-dependent; the same $\delta = .70$ setting is the oracle-best result on S4G but is harmful on S8P.

E. Operational Interpretation

The sensitivity results reveal two coupled controls. The pair-adaptation weight changes how much a candidate edge trusts direct record overlap, whereas τ determines how much accumulated evidence is required before transitive clustering. A high λ followed by a high τ is doubly conservative and can fragment noisy entities. Conversely, $\lambda = 0$ combined with a low threshold can allow generic blocks to create large false-positive components. The best S8P settings illustrate this difference: the isolated λ sweep peaks at .75 under its companion settings, while the separate threshold sweep reaches the dataset's higher oracle F1 at $\tau = .05$.

Recursive depth and filtering act earlier by determining which local conjunctions reach the graph, but later thresholds can mask their impact. The logs therefore support a configurable family, not a fixed ranking of stages. Shallow recursion and moderate pair weighting are reasonable starting points, but this is not a validated label-free tuning rule.

F. Failure Patterns in the Existing Logs

The aggregation reports also classify the direction of errors. On S12PX, 182 of the 185 logged evaluations are reported as false-negative dominant, indicating that the hardest dataset is primarily recall-limited. This agrees with its mean truth-cluster size of 8.66 and the improvement obtained from a low edge threshold. In contrast, 74 of the 185 S1G evaluations are reported as false-positive dominant. With only 50 records and small truth clusters, permissive settings can link unrelated records even when recall is already high.

The logs identify several zero-F1 or near-zero regimes, including thresholds that exceed the useful edge-weight scale, top-$k = 1$ settings that remove most redundant co-occurrence evidence, and degenerate composite-filter weights. These cases are useful because they expose interactions that a single best-score table would hide. A failure at high τ is a graph-pruning failure, whereas a failure at very small k occurs before edge weighting. The distinction reinforces the value of stage-level diagnostics: two configurations can produce the same final F1 for different reasons and therefore require different corrections.

TABLE VIII
BEST OBSERVED F1: DWM VS. THIS WORK (ORACLE SELECTION)

	DWM [4]			This work			
Data	Prec.	Rec.	F1	Prec.	Rec.	F1	ΔF1
S1G	1.000	.963	.981	.964	1.000	.982	$+.001$
S2G	1.000	.896	.945	.917	.917	.917	$-.028$
S4G	.985	.887	.934	.941	.831	.883	$-.051$
S5G	.991	.873	.928	.873	.900	.886	$-.042$
S7GX	.946	.867	.905	.871	.903	.886	$-.019$
S8P	.788	.888	.835	.553	.506	.528	$-.307$
S12PX	.783	.772	.777	.664	.306	.419	$-.359$
S14GX	.958	.823	.892	.923	.839	.879	$-.013$

G. Comparison with the DWM Legacy System

Talburt et al. [4] evaluated the Data Washing Machine (DWM) on 18 person-record samples drawn from the same underlying corpora used here. Eight of those samples correspond directly to the datasets in this study (S1G, S2G, S4G, S5G, S7GX, S8P, S12PX, S14GX). DWM operates by frequency-based blocking followed by all-pairs scoring within each block using a multi-token scoring matrix, entropy-based cluster acceptance, and iterative threshold increases. The present method replaces explicit pairwise scoring with pair-adaptive graph weighting computed during block-to-edge expansion. Table VIII reports the best published DWM result alongside the best logged result from this study for each shared dataset.

On the six good-quality datasets the gap is small: the mean F1 difference is -0.025 and the present method matches or exceeds DWM on S1G. These results indicate that recursive co-occurrence blocking with pair-adaptive graph weighting recovers most of the matching quality that DWM achieves with an additional pairwise scoring stage, at least when data quality is high. The precision–recall balance differs: DWM achieves higher precision on good-quality datasets (mean 0.980 versus 0.898), while this work compensates with higher recall on several datasets (S1G, S5G, S7GX).

The poor-quality datasets S8P and S12PX show a much larger gap (-0.307 and -0.359 respectively). DWM's pairwise Monge-Elkan scorer with normalized edit-distance similarity can align tokens across noisy variants that share few exact

tokens, sustaining high recall even when blocking evidence is diffuse. The present method depends on Jaccard token overlap for pair weighting, which degrades when true-match pairs share fewer verbatim tokens due to data quality errors. This gap identifies the primary limitation of a blocking-only pipeline on low-quality data and motivates future integration of lightweight pairwise signals at the graph-weighting stage.

Despite this divergence on low-quality data, the two systems yield broadly comparable F1 across the majority of tested conditions. The mean F1 difference on the six good-quality datasets is only -0.025, and the present method matches DWM on S1G. The critical distinction is therefore one of pipeline complexity rather than output quality. DWM requires an explicit pairwise scoring pass within every block, evaluating each candidate pair against a full token-by-token scoring matrix at $O(|T|^2)$ cost per pair, and repeating the process iteratively at increasing match thresholds. The present method subsumes pair-level evidence into the graph-weighting step via Jaccard similarity computed in $O(|T|)$ per edge during block expansion, with no separate matching iteration. Both methods produce similar cluster outputs on high-quality data; the added cost of DWM's pairwise stage is most justified on poor-quality data where token-overlap signals alone are insufficient.

H. Limitations

The evaluation has five important limitations. First, all datasets contain person records with closely related schemas, so conclusions should not be generalized to products, citations, or web entities without further evidence. Second, the parameter study changes named axes rather than exhaustively evaluating their Cartesian product. Third, per-dataset best results use oracle ground-truth selection; a deployment without labels still needs a fixed configuration or a label-free selection criterion. Fourth, the current logs emphasize final pairwise clustering metrics and do not separately report standard blocking measures such as pair completeness and reduction ratio. The paper therefore supports the proposed components and their observed sensitivity, and demonstrates competitiveness with DWM on good-quality data, but does not claim universal superiority over all prior systems.

VI. Conclusion

This paper presented an interpretable ER framework that recursively grows token block keys and adaptively modulates block-induced edges with pairwise Jaccard similarity. Across the existing eight-dataset parameter study, pair adaptation produces the clearest precision–recall shift, low graph thresholds are necessary on poor-quality large-cluster data, and deeper recursion yields diminishing F1 returns. Comparison with the DWM legacy system [4], which couples frequency-based blocking with exhaustive pairwise Monge-Elkan scoring, shows that the blocking-only pipeline is within 0.05 F1 on all six good-quality datasets and matches DWM on S1G, but trails by more than 0.30 F1 on the two poor-quality datasets where edit-distance pairwise scoring is decisive. The best logged configuration varies across datasets, confirming that the framework exposes meaningful operating trade-offs. Future work should evaluate fixed label-free parameter selection, blocking-specific metrics, additional domains, and lightweight pairwise signal integration to close the remaining gap on noisy data.

Acknowledgment

This research was partially supported by the National Science Foundation under EPSCoR Award No. OIA-1946391.

References

[1] H. B. Newcombe, J. M. Kennedy, S. J. Axford, and A. P. James, "Automatic linkage of vital records," *Science*, vol. 130, no. 3381, pp. 954–959, 1959.

[2] I. P. Fellegi and A. B. Sunter, "A theory for record linkage," *Journal of the American Statistical Association*, vol. 64, no. 328, pp. 1183–1210, 1969.

[3] P. Christen, *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Springer, 2012.

[4] J. R. Talburt, A. K. Al Sarkhi, D. Pullen, L. Claassens, and R. Wang, "An iterative, self-assessing entity resolution system: First steps toward a data washing machine," *International Journal of Advanced Computer Science and Applications*, vol. 11, no. 12, pp. 680–689, 2020.

[5] M. A. Hernandez and S. J. Stolfo, "The merge/purge problem for large databases," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 1995, pp. 127–138.

[6] S. Yan, D. Lee, M.-Y. Kan, and C. L. Giles, "Adaptive sorted neighborhood methods for efficient record linkage," in *Proceedings of the 7th ACM/IEEE-CS Joint Conference on Digital Libraries*, 2007, pp. 185–194.

[7] U. Draisbach and F. Naumann, "A generalization of blocking and windowing algorithms for duplicate detection," in *Proceedings of the International Conference on Data and Knowledge Engineering*, 2011, pp. 18–24.

[8] G. Papadakis, E. Ioannou, and C. Niedereé, "Hierarchical approach to eliminate superfluous comparisons in meta-blocking," in *Proceedings of the EDBT/ICDT Workshops*, 2014.

[9] G. Papadakis, E. Ioannou, E. Thanos, and T. Palpanas, "The four generations of entity resolution," *Synthesis Lectures on Data Management*, 2020.

[10] M. Bilenko and R. J. Mooney, "Adaptive duplicate detection using learnable string similarity measures," in *Proceedings of the 9th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2006.

[11] V. Efthymiou, O. Hassanzadeh, M. Rodriguez-Muro, and V. Christophides, "Matching web tables with knowledge base entities: From entity lookups to entity embeddings," in *Proceedings of the International Semantic Web Conference*, 2019.

[12] D. Zhang, Y. Suhara, J. Li, T. Rekatsinas, and W.-C. Tan, "Autoblock: A hands-free blocking framework for entity matching," in *Proceedings of the VLDB Endowment*, 2020.

[13] S. Thirumuruganathan, N. Li, J. Kang, N. Suganthan G. C., G. Das, M. Ouzzani, and N. Tang, "Deep learning for blocking in entity matching: A design space exploration," in *Proceedings of the VLDB Endowment*, 2021.

[14] J. Wang, G. Li *et al.*, "Ditto: A simple and efficient approach for entity matching," in *Proceedings of the VLDB Endowment*, 2023.

[15] J. R. Talburt *et al.*, "Case count metric for comparative analysis of entity resolution results," *Frontiers in Big Data*, vol. 9, p. 1736939, 2026.